Day 9: Windows Log Hunt

Inspecting Security, System, and Application Logs for SOC-Relevant Events

Field	Details
Intern Name	Amrita Pathak
Date	11 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Why This Task Matters 4
5. Cross-Log Investigation: Filtering, Selecting, and Reading Events 5
5.1 Security Log 5
5.2 System Log 9
5.3 Application Log 12
6. Consolidated Event Summary 17
7. Findings 17
7.1 Reading the Subject field prevented a factual error 17
7.2 An assumption about Event 86 was wrong until the message was read directly 17
7.3 High-value event types can still be low-priority individual events 17
7.4 Filtering strategy should match the log being examined 17
8. Conclusion 18

Executive Summary

This task inspected all three of Windows' core event logs, Security, System, and Application, on the Windows 11 lab endpoint, rather than focusing on a single log or event type as in earlier tasks. Five events were selected, filtered by time range and by event source rather than by a single Event ID, and each was evaluated for its actual usefulness to a SOC analyst during an investigation.

The five events span two from Security (4672, Special privileges assigned to new logon; 4798, a user's local group membership was enumerated), one from System (7045, a new service was installed), and two from Application (1000, an application crash; 86, a certificate enrollment failure). All five events, on this occasion, reflect routine, explainable system behavior rather than any indicator of compromise. Every event's Subject or context was verified before being described, which surfaced two corrections worth noting: a 4672 event that could easily be mistaken for a human admin logon was confirmed to originate from the built-in SYSTEM account, and an initial guess about a recurring certificate error being caused by the machine not being domain-joined was replaced with the actual cause once the raw event message was read directly, a TPM attestation key enrollment attempt against a Microsoft Azure endpoint that does not exist for this VM.

The central finding of this task is procedural rather than about any single event: raw Event IDs alone are not enough to judge usefulness. Every event required reading its Subject, source, and message content in full before its value to an investigation could be honestly assessed, and two of the five events would have been mischaracterized if that step had been skipped.

1. Objective

Inspect the Windows Security, System, and Application logs on the Windows 11 lab endpoint, select at least five events spanning different categories, filter them by time or by event source rather than browsing unfiltered, and evaluate each one for whether it would genuinely be useful to a SOC analyst during a real investigation.

2. Tools and Environment

Item	Detail
Endpoint used	Windows 11 x64 lab VM (DESKTOP-AUHDFVM)
Account used	CyberAmy (local account)
Log viewing tool	Windows Event Viewer (eventvwr)
Logs examined	Windows Logs > Security, System, Application
Filtering methods used	Custom time range (Security); Event source (System); Event level (Application)
Events selected	4672, 4798 (Security); 7045 (System); 1000, 86 (Application)

3. Key Terms

- **Event source**: the specific component or service that generated an event, recorded as a separate field from the Event ID. The same Event ID can carry different meaning depending on its source, and filtering by source narrows a log to only events written by one particular component,

which is especially useful in the System and Application logs where many unrelated components share the same log.

- **Event level**: a severity classification Windows assigns to every event: Critical, Error, Warning, Information, or Verbose. Filtering by level is most useful in logs like Application, where the overwhelming majority of entries are routine Information-level noise and the handful of Error or Warning entries are where genuine issues surface.

- **Task Category**: a field grouping an event under a broader activity type, such as Logon, User Account Management, or Application Crashing Events. It provides a quick sense of what kind of activity an event belongs to before reading its full detail.

- **Persistence (security concept)**: a technique where an attacker ensures continued access to a compromised system even after a reboot or logoff. Installing a Windows service, the subject of Event 7045 in this report, is one of the most common persistence techniques in real intrusions, which is why that event type carries outsized investigative value even when, as here, the specific instance observed is legitimate.

- **Reconnaissance (security concept)**: the stage of an intrusion where an attacker gathers information about the system or environment they have gained access to, such as checking which groups an account belongs to. Event 4798 in this report maps directly to a named reconnaissance technique, which is why it was selected despite this specific instance being routine.

4. Why This Task Matters

The tasks completed so far in this internship each went deep into one log and one event type. Real SOC work rarely affords that luxury. An analyst's day typically involves moving across many different log sources, most of which are producing routine, harmless activity, and the actual skill being tested is efficient triage: knowing which log to check for a given kind of question, how to filter it down quickly, and how to tell, event by event, whether something is worth a second look. The benefits this task specifically builds are worth stating explicitly, not just implied:

- **Cross-log fluency**: A real SOC analyst cannot afford to only know Security events well. System reveals machine health and persistence attempts, Application reveals crashes and installer activity. An analyst who only checks one log misses whatever evidence lives in the other two.

- **Filtering judgment, not just filtering ability**: Knowing the Filter Current Log dialog exists is not the same skill as knowing which field, time, source, or level actually narrows a given log efficiently. That judgment differs by log, and choosing wrong wastes real investigative time.

- **Resistance to false alarms**: Several events in this task look concerning from their Event ID or Task Category alone, a privileged logon, a certificate failure, a service installation, but turn out to be routine once the underlying fields are read. An analyst who reacts to labels rather than evidence generates noise that erodes trust in every future alert they raise.

- **Resistance to false reassurance**: The opposite failure is just as dangerous. Event types like 4798 and 7045 map to real, named attack techniques. Dismissing them automatically because

most instances are benign is how a genuine attack gets missed. The correct habit is checking every time, not assuming either way.

- **Verification over assumption**: Two corrections were made while producing this report, described in the findings, that exist only because the actual event content was checked rather than inferred from an Event ID or source name. That discipline is the single most transferable skill this task develops.

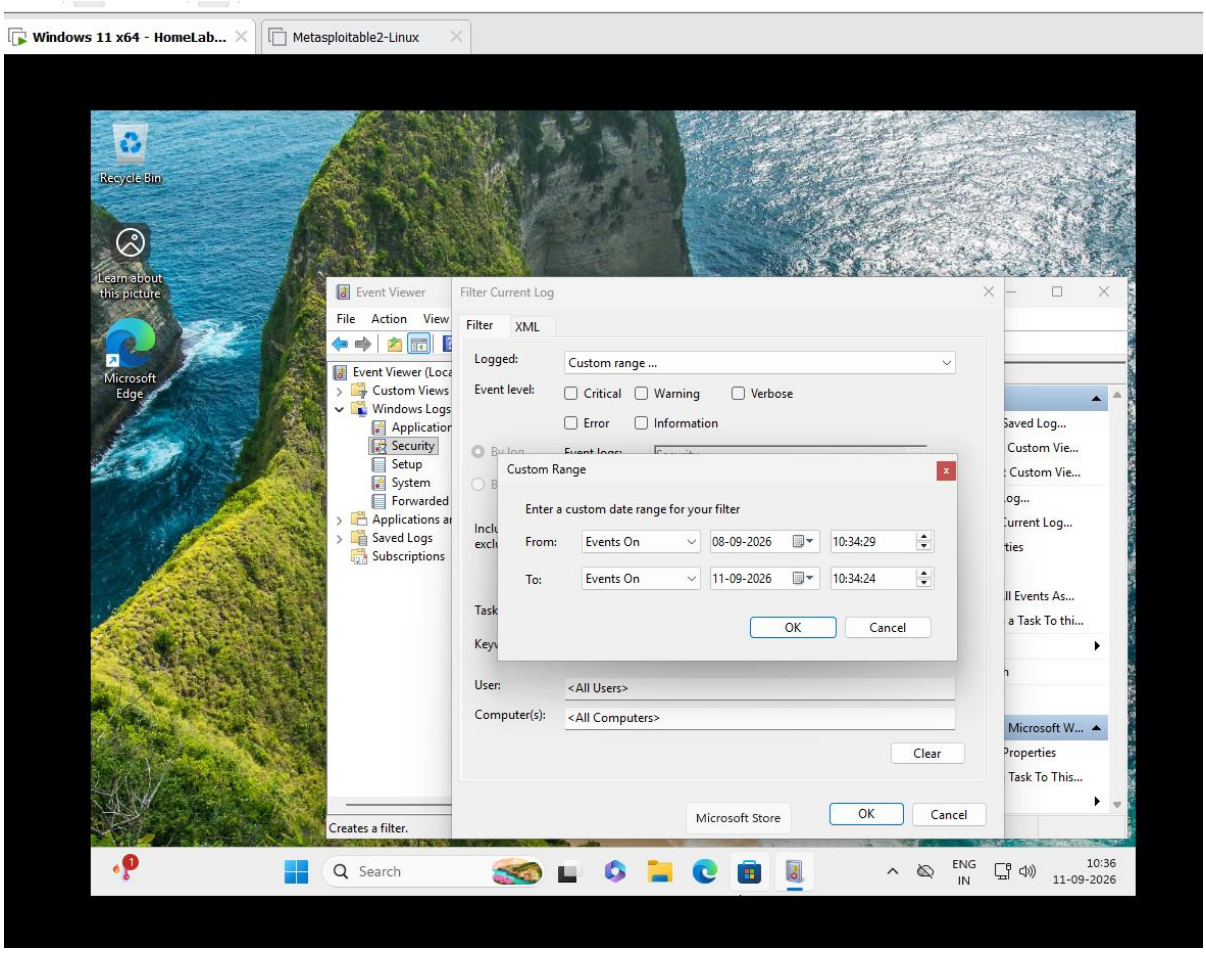
5. Cross-Log Investigation: Filtering, Selecting, and Reading Events

This section documents the actual working process used to move from an empty Event Viewer window to five verified, evidenced events, one log at a time, in the order the investigation was actually carried out.

5.1 Security Log

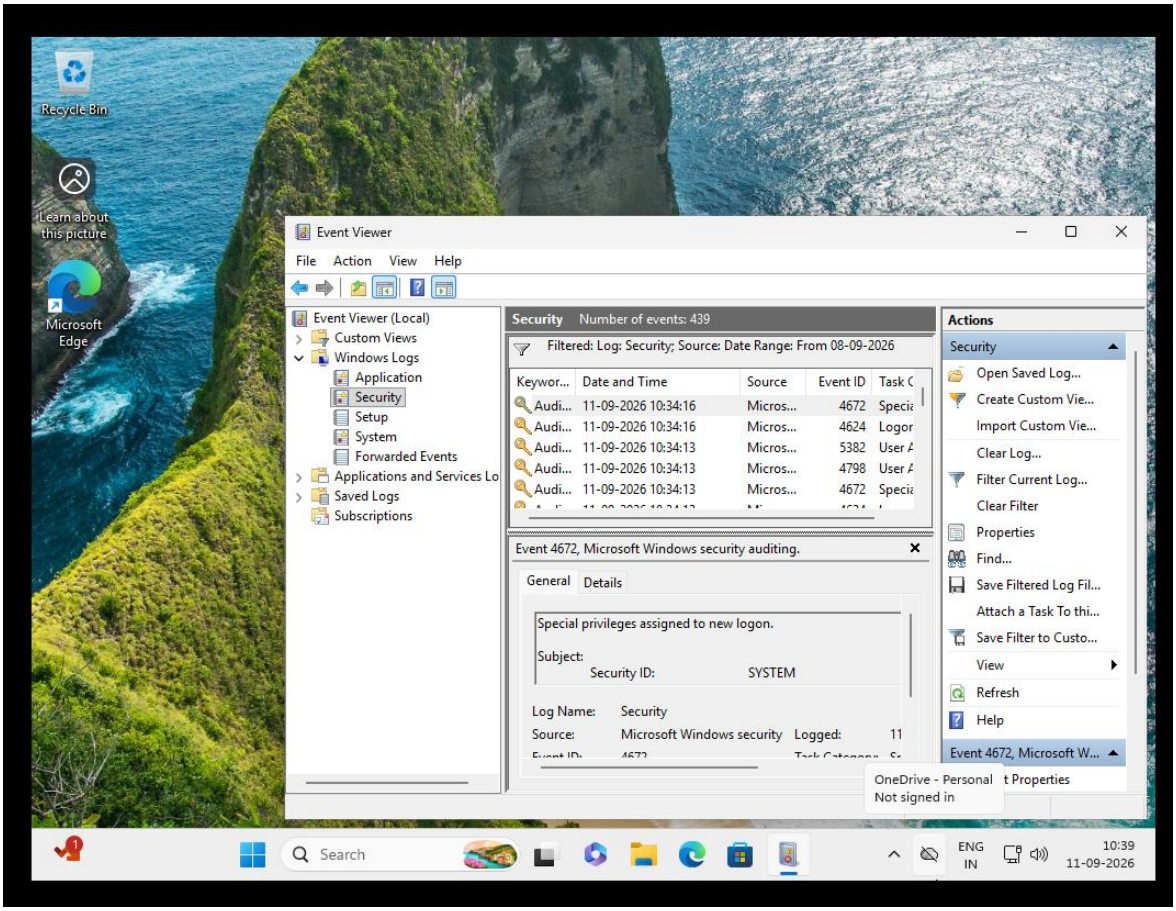
Steps taken, in order:

- **Step 1 - applied a custom time filter**: opened Filter Current Log on the Security log and set a custom date range from 08-09-2026 to 11-09-2026, covering this internship's recent lab activity, rather than filtering by a single Event ID as in earlier tasks. This was a deliberate choice to surface a broad, varied sample rather than one known event type.



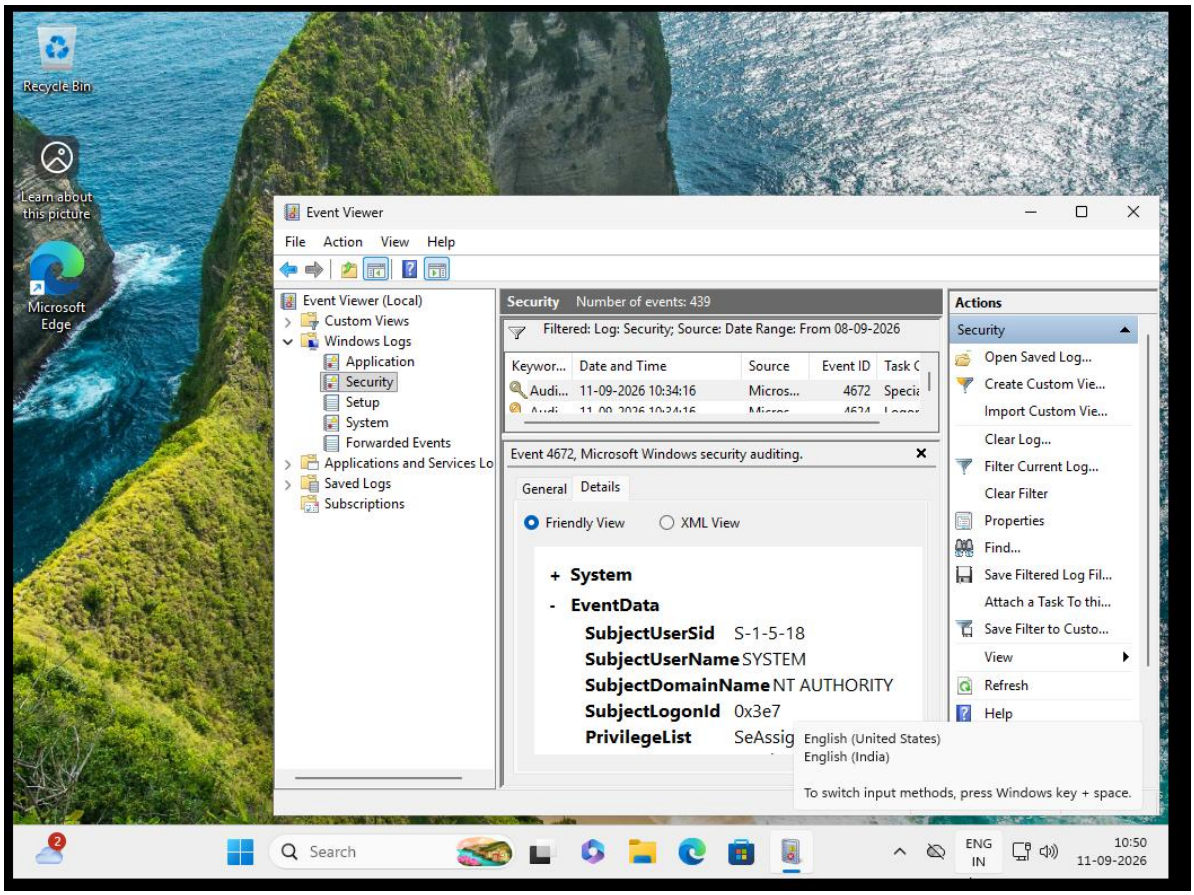
Custom date range filter applied to the Security log, spanning the internship's recent lab activity.

- **Step 2 - reviewed the filtered results**: the filter returned 439 events. Rather than picking the first or most frequent entries, the list was scanned specifically for variety, event types not already documented in Day 6 through Day 8, which ruled out picking another 4624 or 4625.



The filtered Security log with the first selected event, 4672, open in the General tab.

- **Step 3 - selected and read Event 4672 in full**: opened both the General and Details tabs for the first 4672 entry to read its Subject fields before drawing any conclusion.

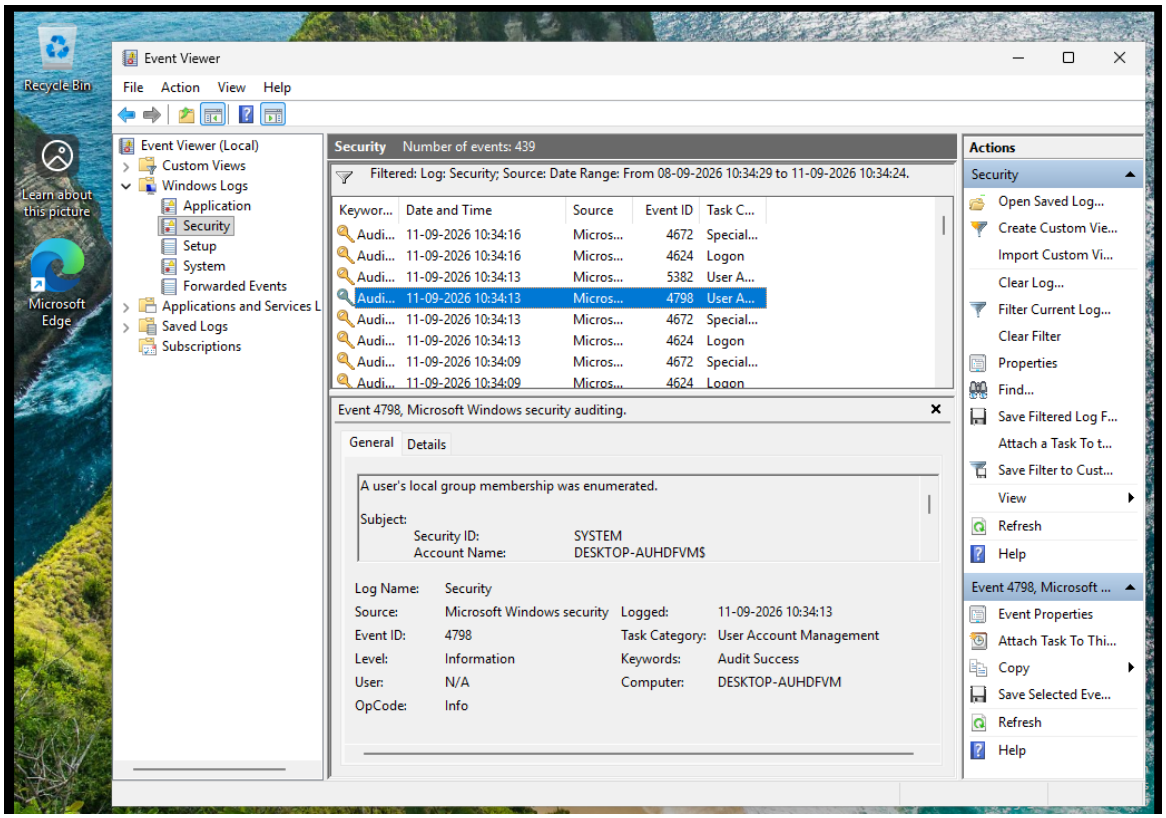


Details tab for Event 4672, showing SubjectUserSid S-1-5-18, SubjectDomainName NT AUTHORITY, and SubjectLogonId 0x3e7.

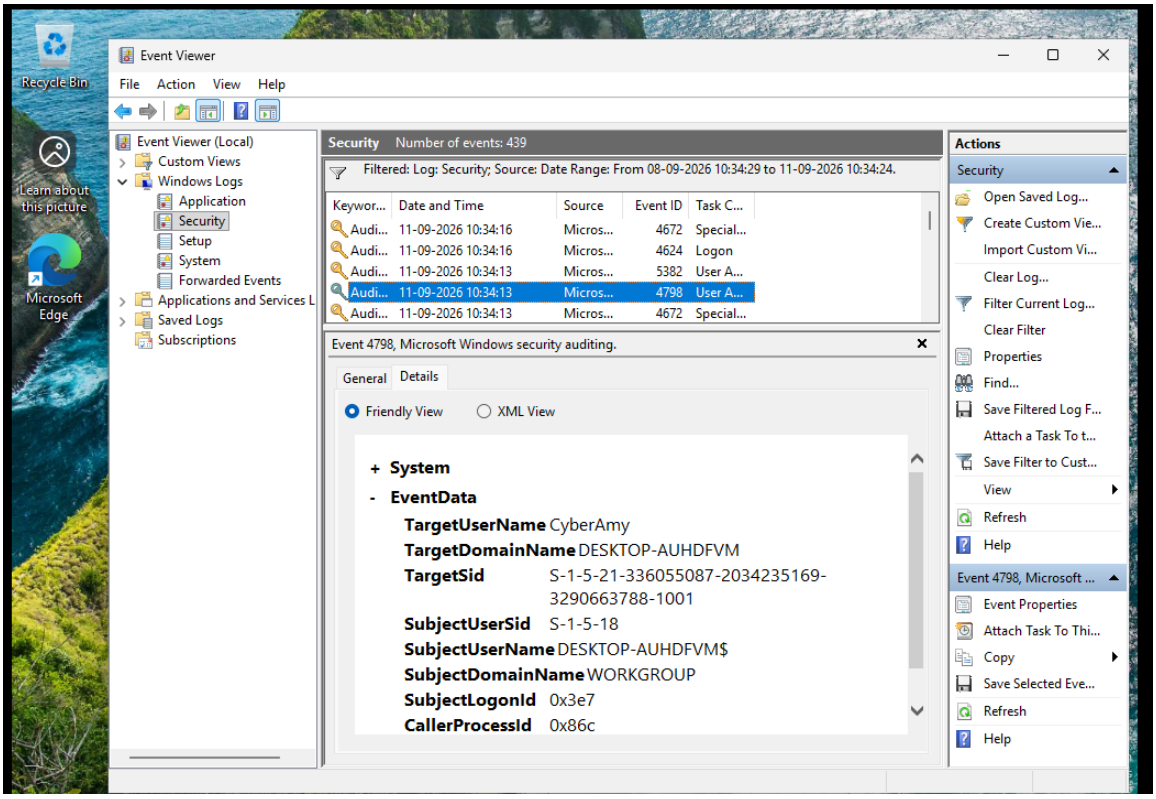
What this showed: SubjectUserSid is S-1-5-18, the well-known SID for the SYSTEM account, which the General tab separately displays as Security ID: SYSTEM. SubjectDomainName is NT AUTHORITY and SubjectLogonId is 0x3e7, the same reserved logon ID pattern used by SYSTEM across earlier tasks. This is not a human administrator logging in, it is Windows' own background operation requiring elevated rights, which is routine and expected on any running machine. A second 4672 entry in the same window was checked as well and traced to NETWORK SERVICE, another built-in service account, confirming the same pattern rather than an isolated case.

SOC usefulness: situationally useful, not useful in this specific instance. Every 4672 in this window traced to a built-in service account, not an interactive human logon, so none of them warrant investigation on their own. The real value of this event type is as a building block: an analyst who knows to check the Subject field before reacting to a 4672 will correctly ignore the SYSTEM and NETWORK SERVICE noise and instead notice the version that matters, a real named user account receiving elevated rights at an unexpected time.

- **Step 4 - selected and read Event 4798 in full**: using the same filtered result set, opened both tabs for a 4798 entry, User Account Management category, timestamped a few seconds apart from the 4672 and 4624 events already reviewed.



Event 4798 selected in the filtered Security log, General tab showing the plain-language summary.



Details tab for Event 4798, showing TargetUserName CyberAmy, SubjectUserName DESKTOP-AUHDFVM$, and SubjectLogonId 0x3e7.

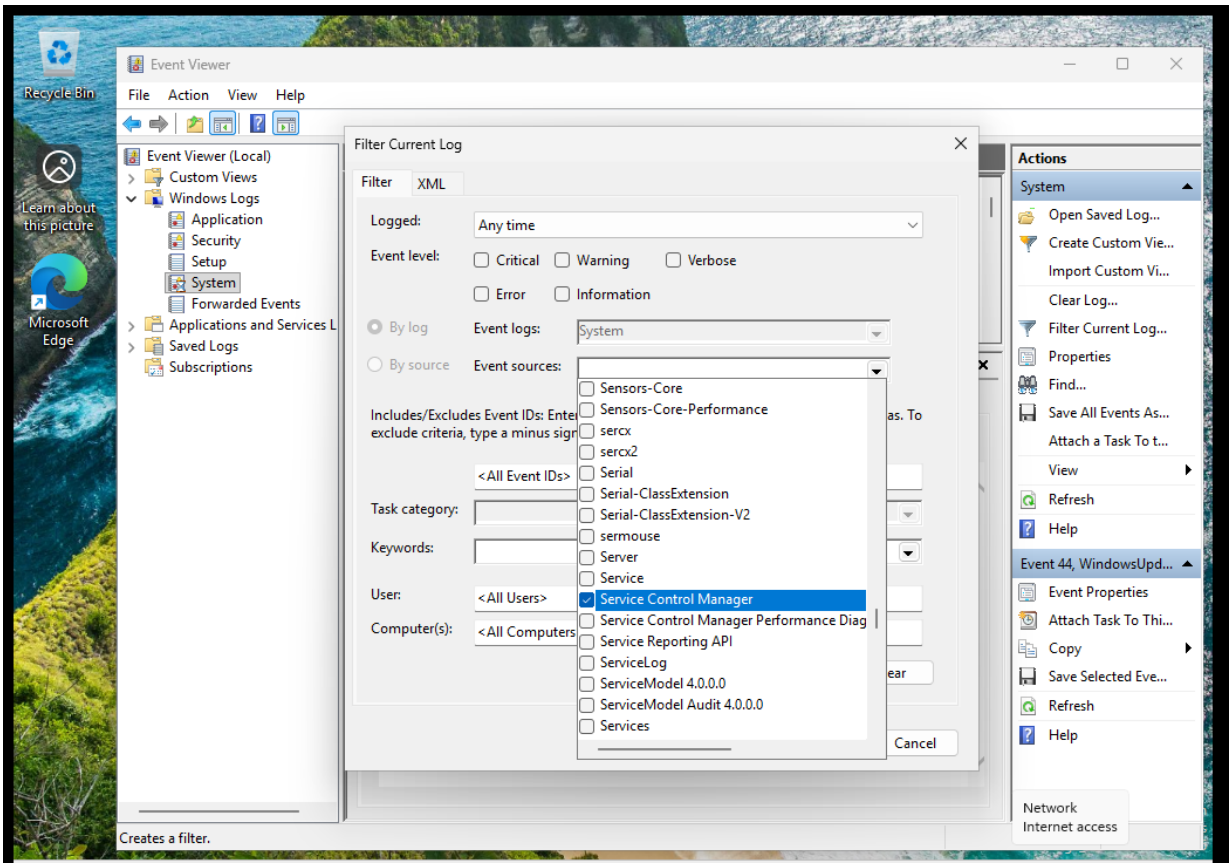
TargetUserName is CyberAmy, this account's own group membership was checked. The raw SubjectUserName field reads DESKTOP-AUHDFVM$, the machine account, which the General tab's Subject line separately resolves and displays as Security ID: SYSTEM, since S-1-5-18 is the SYSTEM SID regardless of which name format a given tab shows. The timestamp, 10:34:13, sits within a few seconds of this session's own 4624 logon and the first 4672 event, which places this as Windows' own logon process routinely determining what CyberAmy's session should be allowed to access, not a separate action by another process.

SOC usefulness: situationally useful. This exact event type maps to MITRE ATT&CK technique T1069, Permission Groups Discovery, a real step attackers take after compromising a machine. In isolation this instance is entirely benign, but the underlying lesson is important: 4798 fires constantly from ordinary logons, so an analyst cannot alert on the event type alone. What separates noise from signal is context, a 4798 firing at logon time from SYSTEM is routine, the same event firing from an unfamiliar process against an account that is not currently logging in would not be.

5.2 System Log

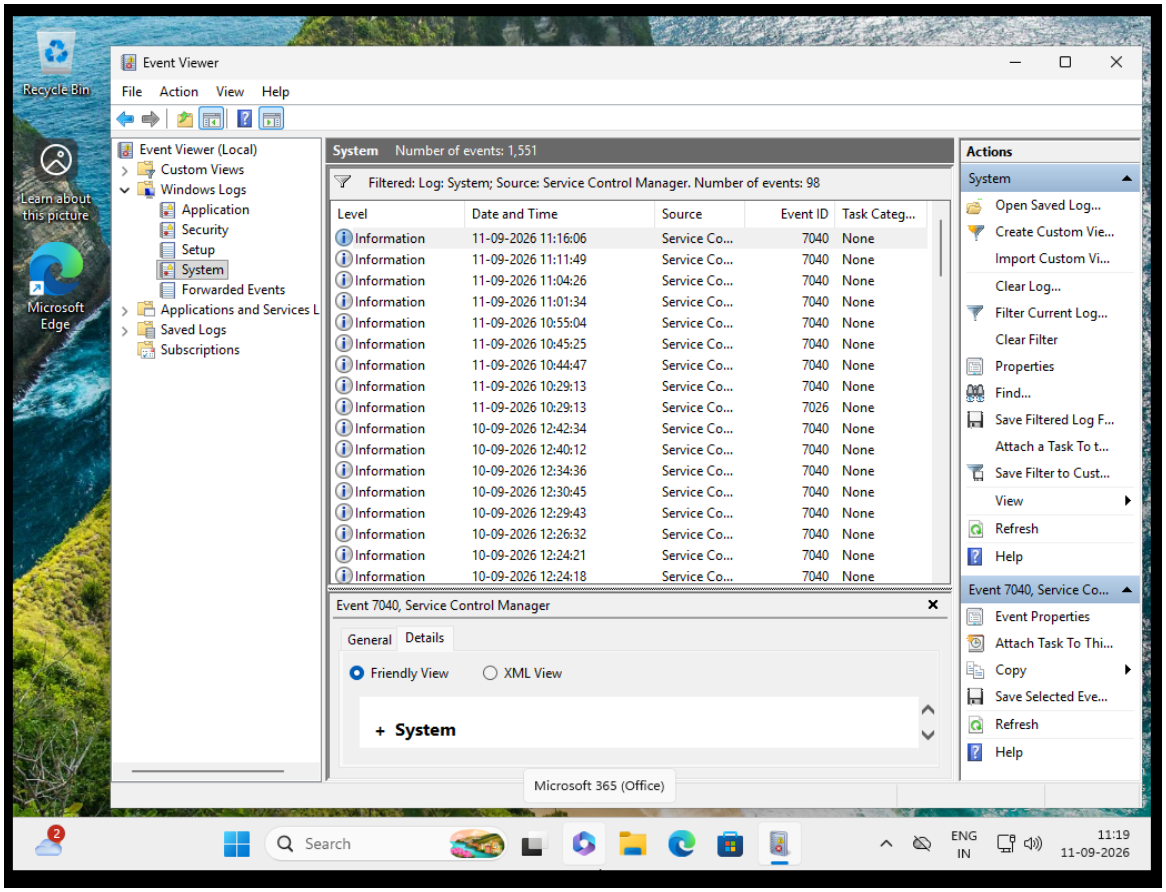
Steps taken, in order:

- **Step 1 - switched the filter approach**: opened Filter Current Log on the System log, this time using the Event sources field rather than a time range, since System log entries are more meaningfully distinguished by which component generated them than by when they occurred. Selected Service Control Manager as the source.



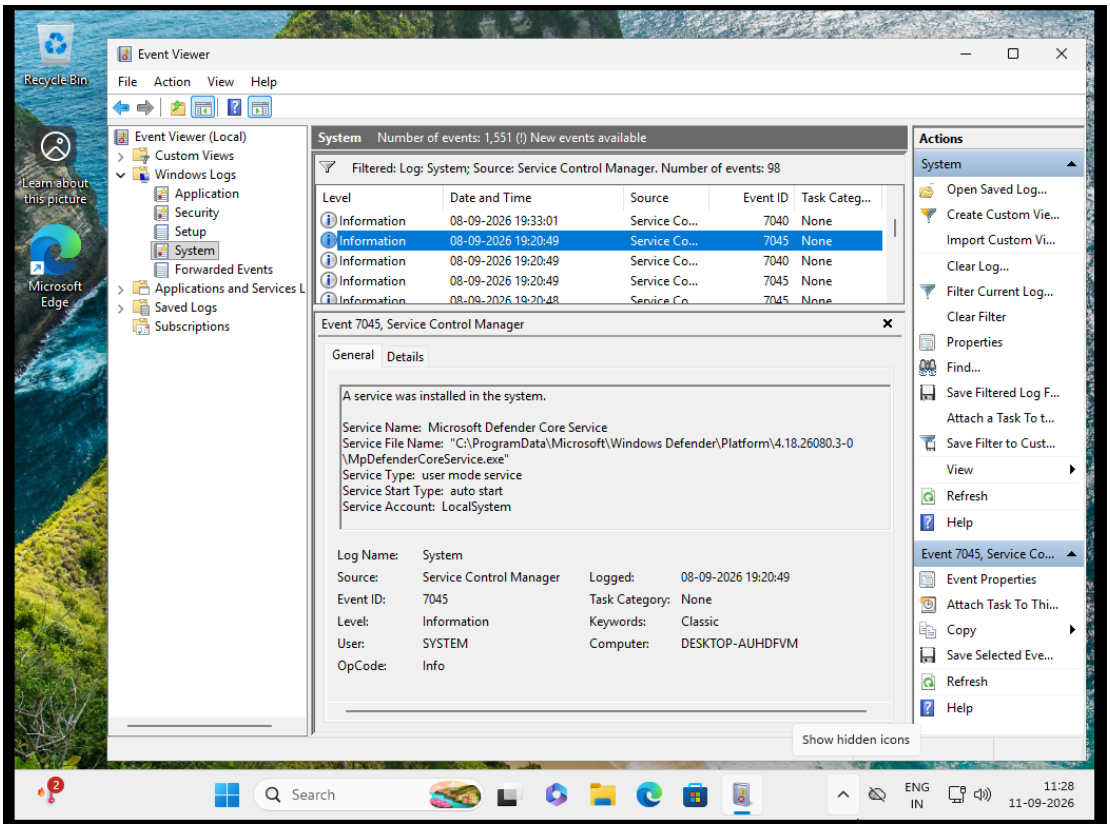
Event sources filter dialog on the System log, with Service Control Manager selected.

- **Step 2 - reviewed the filtered results**: the filter returned 98 events, dominated by repeated 7040 entries (service startup type changes). Scanning the visible Event IDs (7040, 7026, 7045, 7022, 7023), 7045 was identified as the strongest candidate before opening any individual event.

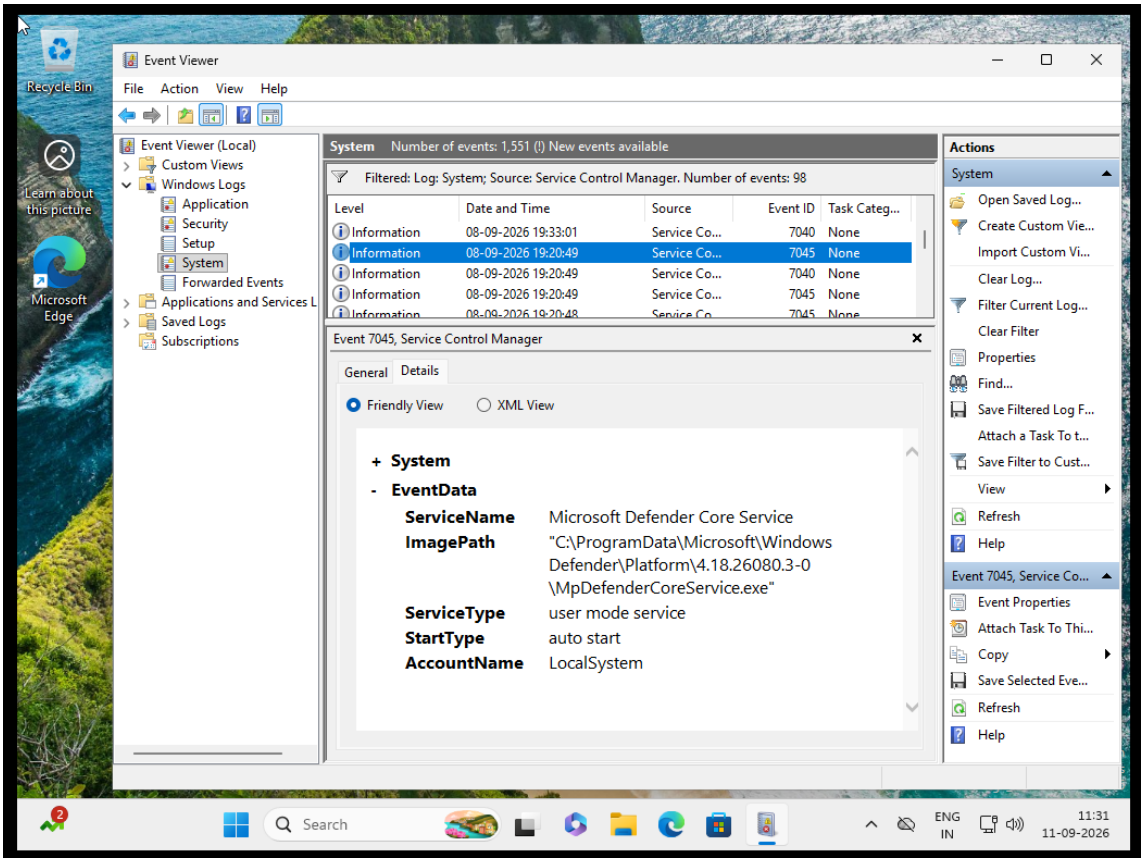


The filtered System log, 98 events from Service Control Manager, before narrowing to a specific 7045 entry.

- **Step 3 - selected and read Event 7045 in full**: opened both tabs for the 7045 entry to confirm exactly what service was installed, by whom, and under what account, rather than treating the Event ID alone as sufficient.



Event 7045, General tab, showing Microsoft Defender Core Service being installed.



Details tab for Event 7045, with the structured ServiceName, ImagePath, StartType, and AccountName fields.

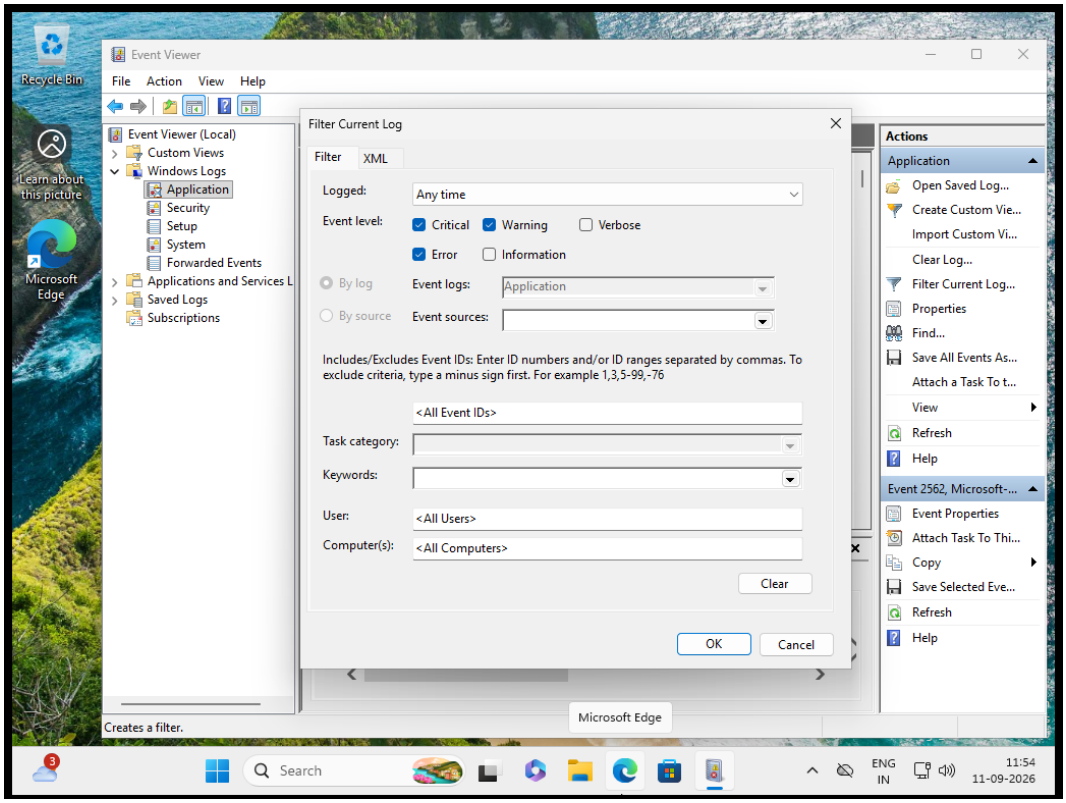
The event records the installation of Microsoft Defender Core Service, running from C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26080.3-0\MpDefenderCoreService.exe, under the LocalSystem account, set to auto start. Every field here is consistent with Defender's normal platform update mechanism, which periodically reinstalls its own core service, a recognizable Microsoft service name, a path under ProgramData\Microsoft, a standard system account, and a start type matching the service's intended purpose.

SOC usefulness: highly useful as an event type, benign in this specific instance. Installing a Windows service is one of the most common persistence techniques used in real intrusions, since a malicious service can run with high privileges and survive a reboot, which is why 7045 maps to a named MITRE ATT&CK technique, T1543.003. This instance is legitimate, but it also establishes a useful baseline: a trustworthy 7045 looks like this one, a recognizable vendor name, a path inside standard system directories, and a matching account and start type. The version an analyst should treat with suspicion is the inverse: an unrecognized or misspelled service name, a path in a user-writable location such as Temp or Public, or an unexpected account.

5.3 Application Log

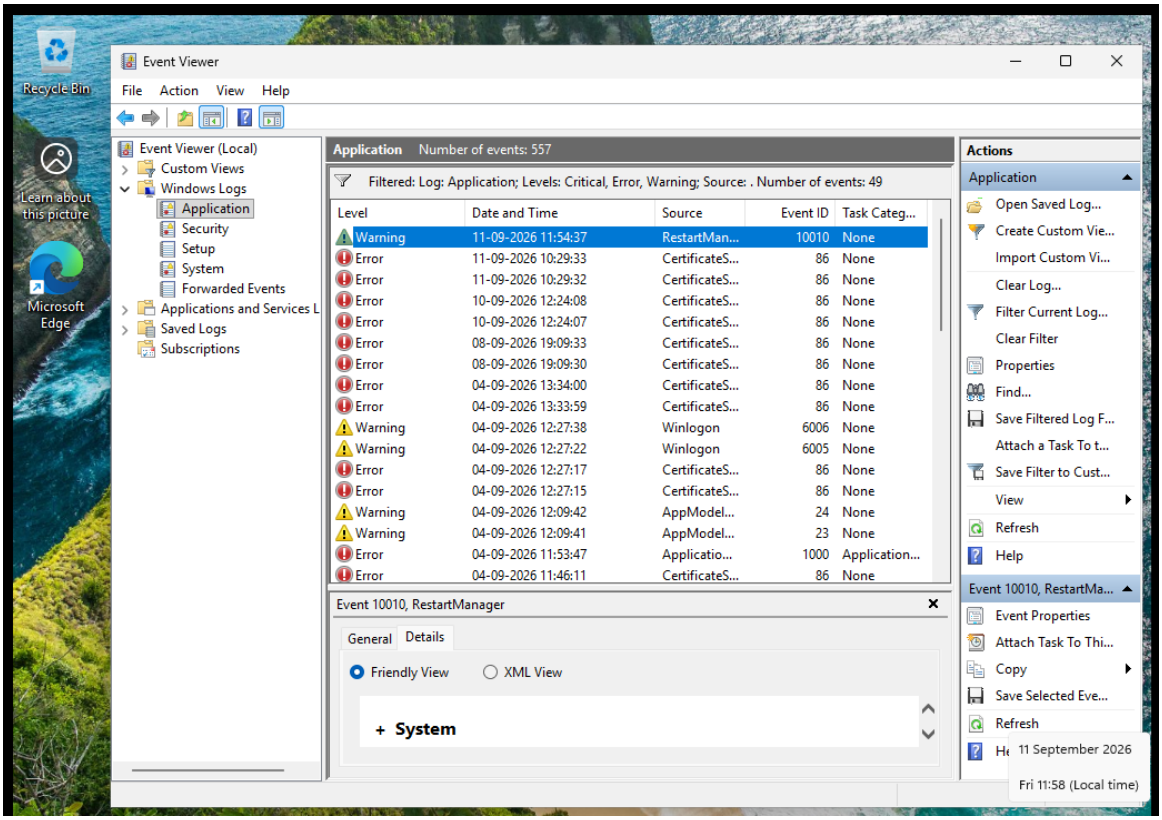
Steps taken, in order:

- **Step 1 - filtered by severity level**: opened Filter Current Log on the Application log and checked Critical, Error, and Warning, leaving Information and Verbose unchecked. This log's Information-level volume, over 500 entries in the same window, consists almost entirely of routine background chatter from licensing, app packaging, and update components, so severity was the most effective first cut here, unlike Security or System.



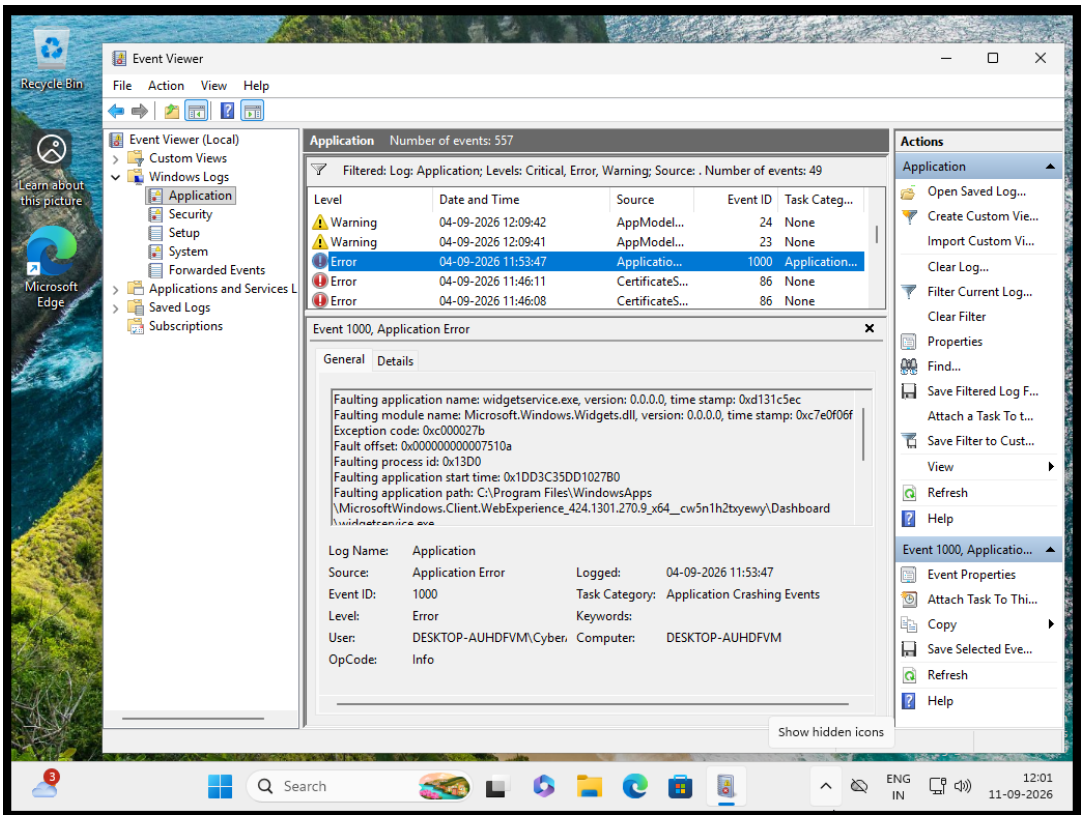
Event level filter applied to the Application log, restricted to Critical, Error, and Warning.

- **Step 2 - reviewed the filtered results**: the filter reduced the log to 49 events. Scanning this smaller set, two candidates stood out: Event 1000, a genuine application crash, and Event 86, an error recurring multiple times rather than a one-off.

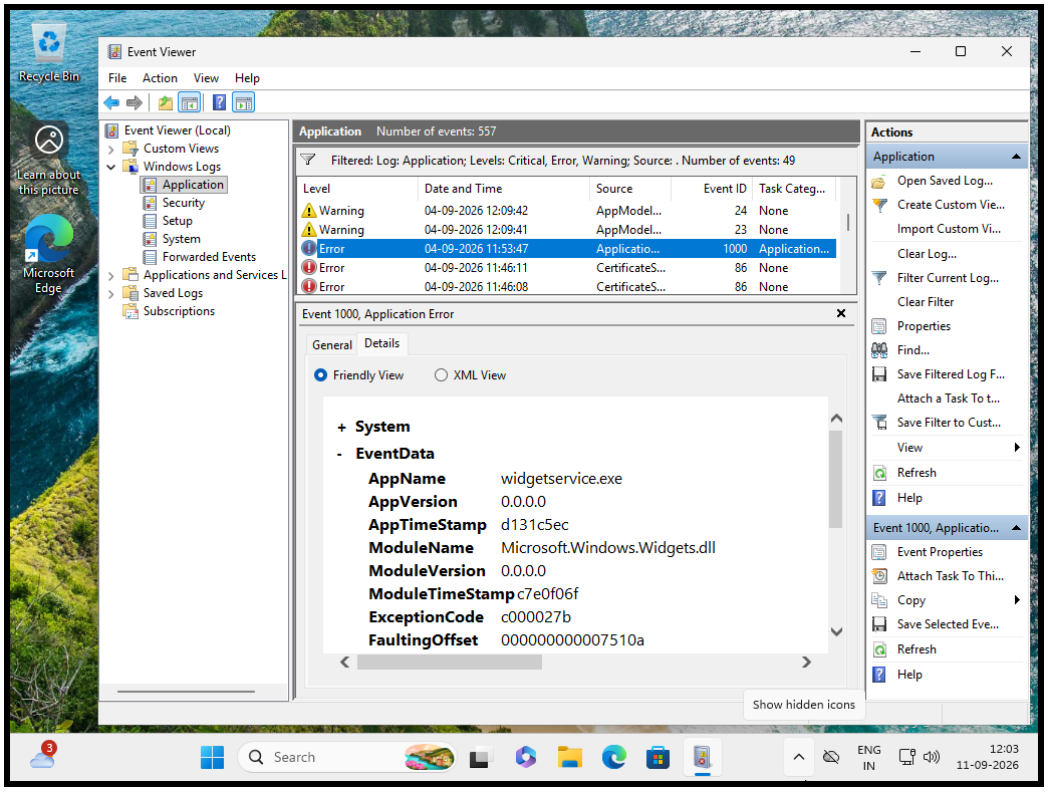


The filtered Application log, reduced to 49 events once Information-level noise was excluded.

- **Step 3 - selected and read Event 1000 in full**: opened both tabs to identify exactly which application crashed and why, rather than treating any Error-level entry as equally significant.



Event 1000, General tab, showing the faulting application and module.

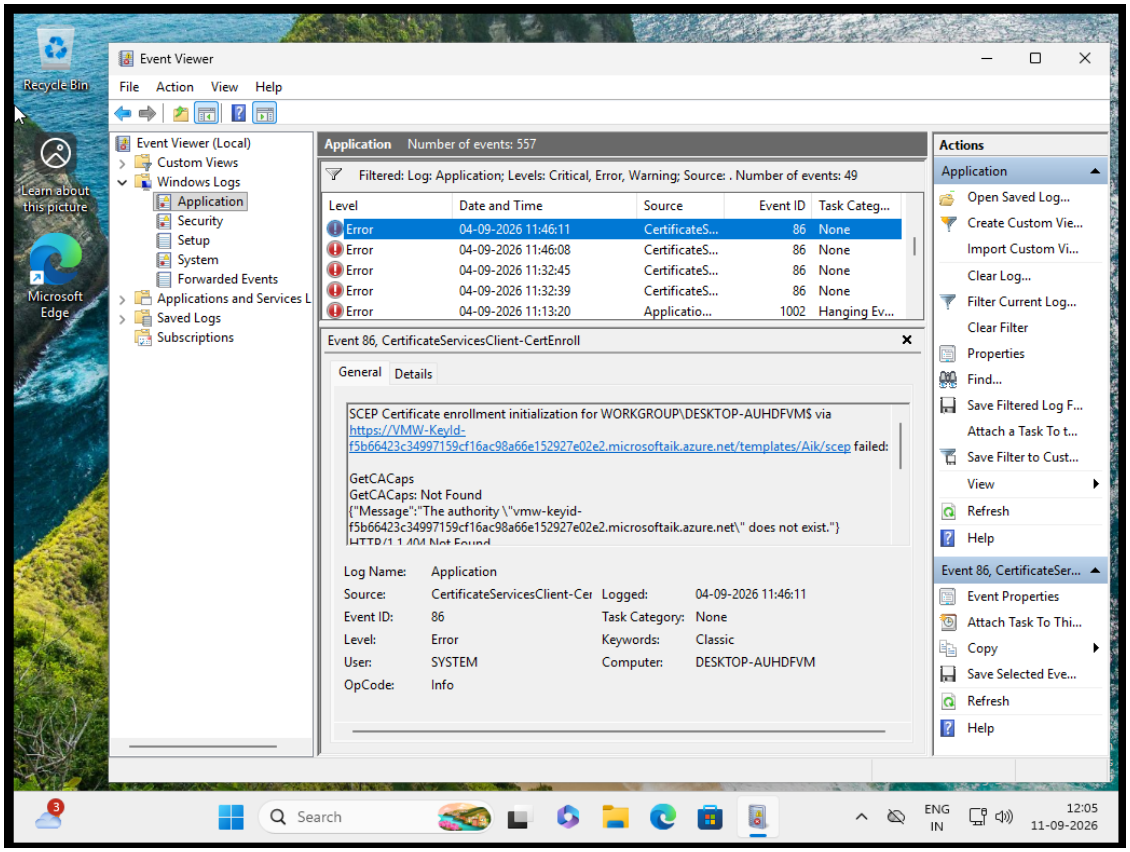


Details tab for Event 1000, with structured AppName, ModuleName, and ExceptionCode fields.

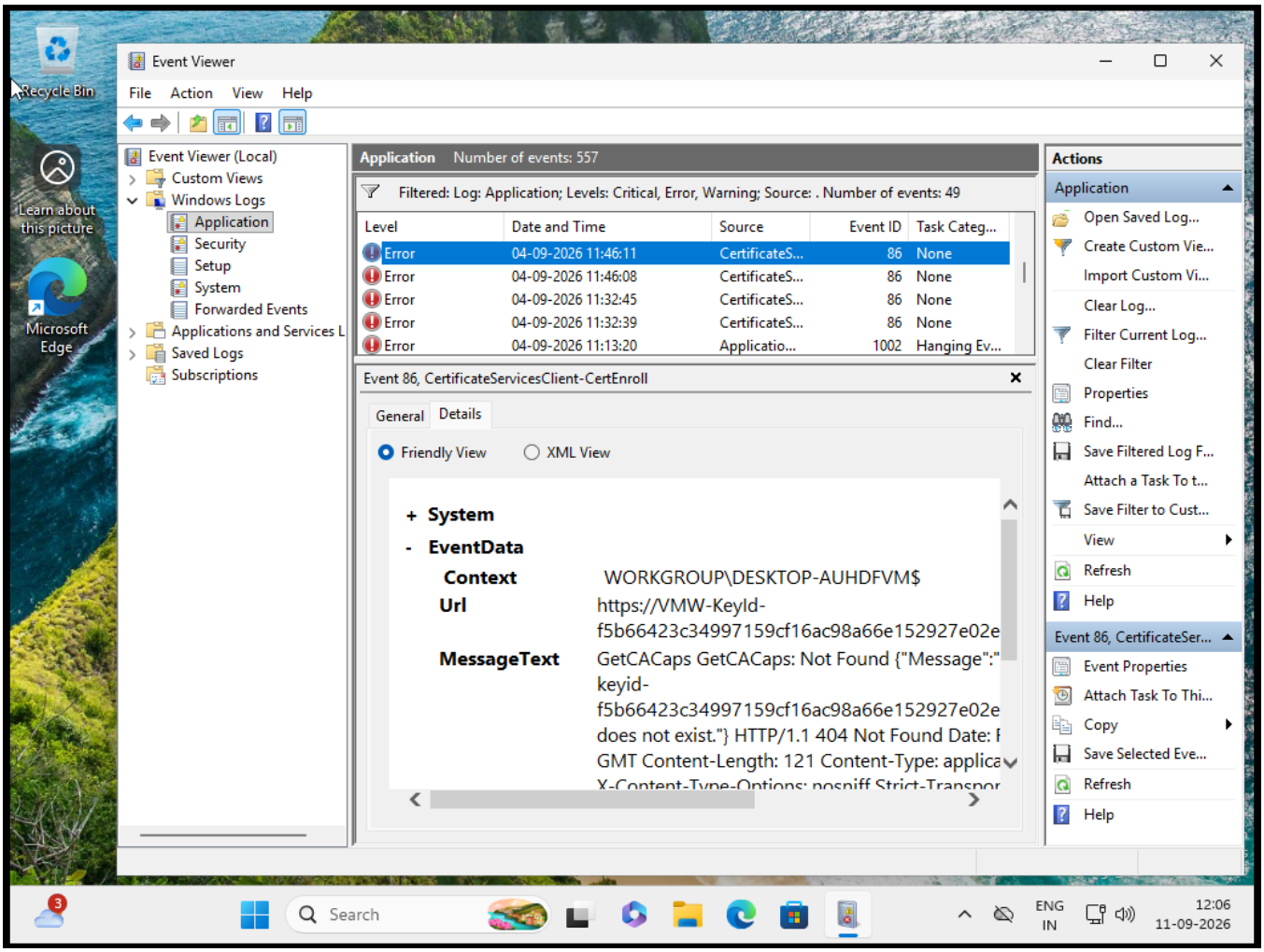
The faulting application is widgetservice.exe, part of the built-in Windows Widgets platform (Microsoft.Windows.Client.WebExperience), crashing in its own module Microsoft.Windows.Widgets.dll with exception code 0xc000027b. This is a widely documented, known issue with the Windows 11 Widgets service rather than anything unusual to this machine. The User field confirms it ran under DESKTOP-AUHDFVM\CyberAmy, this session's own account, at the time of the crash.

SOC usefulness: useful as a category, low priority in this specific instance. A single, well-attributed crash of a known Microsoft component under the logged-in user's own session, with no unusual process path or account involved, is not investigation-worthy on its own. The value of this event type in a real investigation is contextual and cumulative: an analyst correlating a timeline of an incident would want to know if a crash occurred immediately before or after other suspicious activity, since a security tool or monitoring agent crashing at a convenient moment is a recognized pattern worth checking, not because a crash is inherently malicious.

- **Step 4 - selected and read Event 86 in full, then corrected an initial assumption**: opened both tabs for one of the five repeated Event 86 entries. The initial working assumption, based on the source name CertificateServicesClient-CertEnroll alone, was that this reflected a domain Active Directory autoenrollment failure. Reading the full MessageText field directly showed this assumption was wrong, and it was corrected before being written into this report.



Event 86, General tab, showing the full enrollment failure message.



Details tab for Event 86, with the structured Context, Url, and MessageText fields.

The actual message describes a SCEP certificate enrollment attempt for WORKGROUP\DESKTOP-AUHDFVM$ against a URL containing a VMware-specific key identifier (VMW-KeyId), targeting a Microsoft Azure AIK (Attestation Identity Key) endpoint, failing with an HTTP 404 because that specific hostname does not exist. This is a TPM attestation certificate enrollment attempt, tied to this VM's virtual TPM, not a domain Active Directory Certificate Services autoenrollment failure. This same failure recurred five times between 11:13:20 and 11:46:11, roughly 33 minutes, all with an identical cause.

SOC usefulness: low priority, but a useful example of recognizing explainable noise. A recurring identical failure tied to this VM's virtual TPM attempting to reach an Azure attestation endpoint that does not resolve for this environment is a configuration characteristic of the lab setup, not a sign of compromise or tampering. The genuinely useful skill this event demonstrates is not about the event itself, but about the discipline required to read a message in full before writing a conclusion about it, since an Event ID and source name alone were not enough to correctly explain what was happening.

6. Consolidated Event Summary

Event ID	Log	Source	Category	SOC Usefulness
4672	Security	Microsoft Windows security auditing	Special Logon	Situational, not useful in this instance (SYSTEM/NETWORK SERVICE)
4798	Security	Microsoft Windows security auditing	User Account Management	Situational, benign here, maps to T1069 discovery technique
7045	System	Service Control Manager	Service Installation	High value as a type; benign in this instance (Defender)
1000	Application	Application Error	Application Crashing Events	Low priority alone; useful for incident timeline correlation
86	Application	CertificateServicesClient-CertEnroll	None	Low priority; explainable VM/TPM configuration noise

7. Findings

7.1 Reading the Subject field prevented a factual error

The first 4672 event, at a glance, resembles a privileged human logon. Only checking SubjectUserName and SubjectLogonId revealed it was SYSTEM, using the same reserved logon ID pattern seen across multiple earlier tasks. Writing this event up without that check would have produced an inaccurate, more alarming description than the evidence actually supports.

7.2 An assumption about Event 86 was wrong until the message was read directly

Before the raw message was reviewed, the working assumption was that Event 86 reflected a domain Active Directory Certificate Services autoenrollment failure, a reasonable guess given the source name alone. The actual message described something entirely different, a VMware virtual TPM attestation key enrollment attempt against a nonexistent Azure endpoint. This is the clearest example in this task of why an Event ID and source name are a starting point for investigation, never a substitute for reading the actual content.

7.3 High-value event types can still be low-priority individual events

Both 7045 and 4798 map to named, real attack techniques, service installation for persistence and group enumeration for reconnaissance, yet both specific instances examined here were entirely benign. This is not a contradiction. It reflects how real SOC monitoring actually works: certain event types earn a permanent place on a watch list because of what they can indicate, not because every occurrence is a threat. The analyst's job is to apply that context every time, not to alert on the event type alone.

7.4 Filtering strategy should match the log being examined

A single filtering approach did not fit all three logs equally well. Time-range filtering suited Security, where the goal was surfacing whatever occurred during known lab activity. Source filtering suited System, where distinguishing components mattered more than timing. Level filtering suited

Application, where the volume of routine Information-level noise made severity the most effective first cut. Recognizing which filter fits which log is itself a practical skill this task was built to develop.

8. Conclusion

Five events were selected across all three of Windows' core logs, two from Security, one from System, and two from Application, each located through a filtering method suited to that specific log rather than a single blanket approach. Every event was read at the field level, Subject, source, message content, and structured EventData, before any conclusion was drawn about its cause or its value to a SOC analyst.

Two corrections made during this process, the true origin of the 4672 event and the true cause of the recurring Event 86 failure, exist specifically because raw evidence was checked rather than assumed from an Event ID or source name alone. That habit is the actual finding of this exercise: none of the five events required deep technical expertise to read, but all five required the discipline to verify before concluding, which is precisely the discipline a SOC analyst is expected to bring to every alert that crosses their queue.

Together, these five events describe a realistic, low-concern snapshot of ordinary Windows activity, background privilege use, routine logon processing, a legitimate security tool reinstalling its own service, a known application crash, and an explainable certificate enrollment failure tied to this VM's virtual TPM, none rising to the level of requiring escalation, while each still carrying genuine investigative value in the right context.